Data-Oriented Models.

Based on the book by

David William Brown

John Wiley & Sons, ISBN 0471371378

1

1

The need of a middle level notations

- In the last example, when we analyzed a database, database tables are the final results we have
- How can we be sure that the analysis is correct by watching these tables?
- What is the logic of our analysis, if others try to understand our system? Final tables are very bad explanations.
- Can we have some kind of notations that help to record our thoughts or encourage our rethinking of the design?
- What is the essence behind the design logic?

2

2

4.1. Data Models: The ERD

- **Mid-1970s: Databases were just coming into use**

- Research to find ways to model data
- To give template for database design
- Many notations were tried
- Chen (1976) proposed the

Entity-Relationship Diagram (ERD)

3

3

The ERD was the first systems analysis tool to focus on

DATA

and

How it is Linked and Organized.

It was from this that objects then evolved.

4

4

4.1. Data Models: The ERD

- **Chen made two contributions:**
  - The Entity-Relationship Principle, and
  - The Entity-Relationship Notation.
- These further issues were also investigated by Chen and others:
  - E-R models for database design
  - The significance of the data of an enterprise
  - Data as a corporate asset.
  - The stability of the data of an enterprise

5

5

1 The Entity-Relationship *Principle*.

- **To manage a sales operation, we need to keep data about :**

Sales

- **Customers**
- **Products**
- **Sales**
- **Sales clerks**

6

6

1 The Entity-Relationship *Principle*.

- To run a phone company, we would need to keep data about:

Phone
- Customers
- Numbers
- Lines
- Calls
- Services

7

7

1 The Entity-Relationship *Principle*.

- To run a booking agency we must record information about:

Booking
- Venues
- Artists
- Agents
- Concerts
- Performances
- Customers
- Seats

8

8

1 The Entity-Relationship *Principle*.

- To run a booking agency we must record information about:

Booking
- Venues
- Artists
- Agents
- Concerts
- Performances
- Customers
- Seats

- Notice there is no Date in this list.

Why Not?

9

9

1 The Entity-Relationship *Principle*.

- A date is data that describes a Concert
  - A date can describe many things
  - The key word here is

Things

10

10

1 The Entity-Relationship *Principle*.

Each of these lists is a list of *things*

Sales	Phone	Booking
Customers	Customers	Venues
Products	Numbers	Artists
Sales clerks	Lines	Agents
	Calls	Concerts
	Services	Performances
		Customers
		Seats

11

11

1 The Entity-Relationship *Principle*.

- The E-R principle focuses on the things we need to keep data about.
  - Earlier methods emphasized what the users need to know to do their job.
  - Here we emphasize what things the users need to know *about*.
  - The word Entity means a thing.
  - *Later* we worry about what items they need to know *about* each entity.

12

12

1 The Entity-Relationship *Principle*.

- **Webster: An entity is "something that has separate and distinct existence"**
- You can see it is something we need to know about (i.e., keep data about) to do our job
- So we define it. . .

13

13

1 The Entity-Relationship *Principle*.

A Data Entity is some*thing* that has separate and distinct existence *in the world of the users* and is of interest to the users in that they need to keep data *about* it in order to do their job.

14

14

1 The Entity-Relationship *Principle*.

So these are the entity lists for these businesses:

Sales
- Customers
- Products
- Sales clerks

Phone
- Customers
- Numbers
- Lines
- Calls
- Services

Booking
- Venues
- Artists
- Agents
- Concerts
- Performances
- Customers
- Seats

15

15

1 The Entity-Relationship *Principle*.

- **Grammatically, the term Entity refers to a single specific *Customer* or *Product* or *Sale* or *Call* or *Artist*.**
- What we need to talk about mostly is the class of all Customers, or in other words the type of thing called a Customer.

16

16

1 The Entity-Relationship *Principle*.

So we define:

An Entity Type is a class or category expressing the common properties that allow a number of entities to be treated similarly.

17

17

1 The Entity-Relationship *Principle*.

An individual *Customer* or *Product* or *Sale* or *Call* or *Artist* is then an

Occurrence

of the Entity Type.

18

18

- Entity type
- Entity
- Occurrence
- class
- object
- instance

19

19

1 The Entity-Relationship *Principle*.

- I am an occurrence of the entity type "Employee"
- You are an occurrence of the entity type "Student"
- We each live in an occurrence of the entity type "Dwelling"
- The last time you caught an occurrence of the entity type "Bus," it followed an occurrence of the entity type "Route" while executing an occurrence of the entity type "Trip."

20

20

1 The Entity-Relationship *Principle*.

- *Attributes* are the data elements carried by an entity that describe it and record its state.
- *Attributes* are the things we need to know *about* an Entity.

21

21

1 The Entity-Relationship Principle.

Associations:

- So we have:
  - Hundreds of *Customers*
  - Thousands of *Products*
  - Dozens of *Sales Clerks*
- Do we have a business?

NOT YET!

22

22

1 The Entity-Relationship Principle.

Associations:

Not until:

- A Customer buys a Product
  - A Sales Clerk sells a Product
  - A Sales Clerk sells to a Customer
- These are the inter*actions* that occur between entities.
- Note each involves a verb.

23

23

1 The Entity-Relationship Principle.

Associations:

We define:

An Association is the interaction of two entities and is represented by a verb.

24

24

❶ The Entity-Relationship Principle.

Associations :

- The purpose is to provide access paths to the data.
- When a sale occurs, we will link:
  - A Customer
  - To a Product
  - To a Sales Clerk.
- Using a *verb* to describe each link.

25

25

❷ E-R models for database design.

- To start with, the search for data modeling methods was for *database* design.
- Many notations were developed and some actually tried out.
- Chen's ERD turned out best and took over.
- It also caught on as a tool for *systems* analysis, as well as for data analysis.

26

26

❷ E-R models for database design.

For database design:

- Each *Entity Type* becomes a file or table.
- Each *Attribute* becomes a field (i.e., a column)
- Each *Association* becomes an access pathway (i.e., a foreign key)

27

27

A non-intersect history

- In the early 70s' Database field begins to understand the important of data. The idea of entity type (class) occurrence (objects) and attribute begins to emerge. And, more importantly the stability of data
- In the meantime, software engineering enter the era of procedure-oriented, searching for reusable modules and procedures.

28

28

❸ The Stability of Data.

- This year, our favorite Transit Company is concerned with entity types:
  Bus, Driver, Route, Trip, Bus Stop and Passenger.
- Ten years from now, there will be many changes in how the company is run, and in how the people do their jobs.
- But they will *still* have to deal with

Buses, Drivers, Routes, Trips, Bus Stops and Passengers!

- Not only that, but ten years ago they worked with

Buses, Drivers, Routes, Trips, Bus Stops and Passengers!

29

29

❸ The Stability of Data.

- However, entities are neither static nor stagnant;
- There may be an occasional new entity,
- There will often be new attributes,
- But as long as we stay in the

same business

there will be very

few new entities.

30

30

3 The Stability of Data.

This stability contributes greatly to reducing the costs and delays in system maintenance.

31

31

3 The Stability of Data.

However, *beware new business!*

Mergers, acquisitions, takeovers and diversifications can all put your company into

new businesses

with a host of

new entity types.

32

32

4 The Significance of Data.

The Data modeler's Creed:

Data

is the

Centre of the Universe

33

33

In the meantime

Smalltalk Xerox Pal

- SE researchers search for reusable software unit based on procedure oriented had failed !!
- Some bright people began to think the essence of software
- They suddenly aware of that the data is seldom changed but procedures (system functions) are changed frequently

34

34

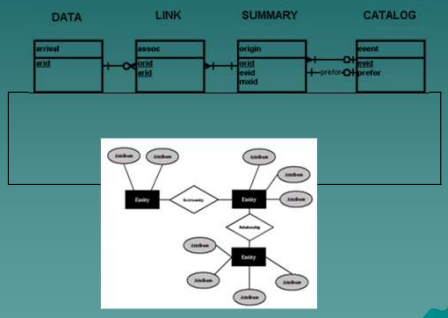
6 The Entity-Relationship Notation.

- There are many E-R notations.
- All of them work, any will do the job.
- Use whichever one you or your boss or your professor or your client prefers.
- UML is now the standard notation in the object world, so
- We will use UML for our Entities as well as our Objects.

35

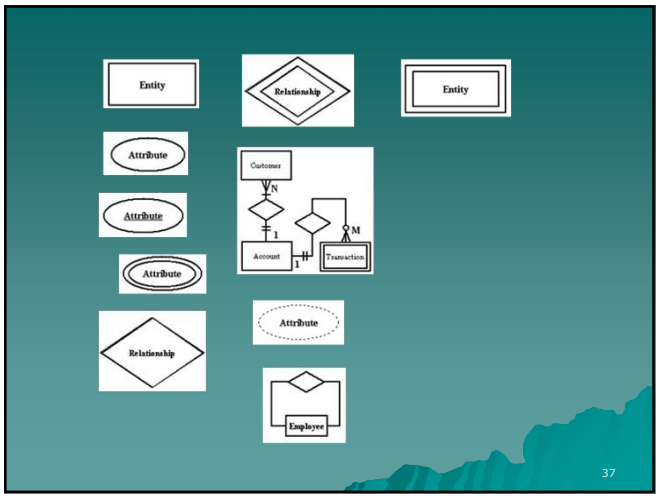
35

ERD diagrams notations



36

36



37

UML era

- Now, UML is the most popular notations for system analysis and design
- So, we use UML to draw ERD

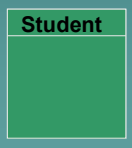
38

38

The Entity-Relationship Notation.

Entities

- Draw an Entity as a square or rectangular box, divided by a line near the top.
- Above the line place the name, singular.



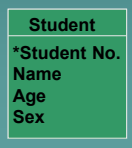
39

39

The Entity-Relationship Notation.

Attributes

- If there are not too many, Attributes may go in the bottom part of the box.
- Primary key first, marked with an asterisk.
- If not enough room, show only the key, list the others in the accompanying write-up.



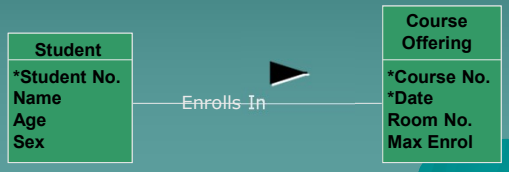
40

40

The Entity-Relationship Notation.

Associations:

- Draw a line joining two entity boxes to show a relationship exists
- Write the verb above the line.



41

41

The Entity-Relationship Notation.

Associations

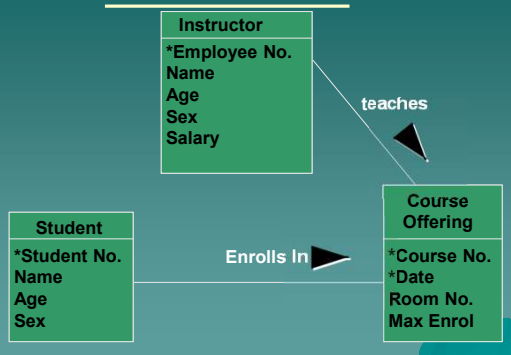
- Add a solid arrowhead so that it makes a sentence when you read it in the direction of the arrow:
- "Student *enrols in* course"

42

42

6 The Entity-Relationship Notation.

Associations

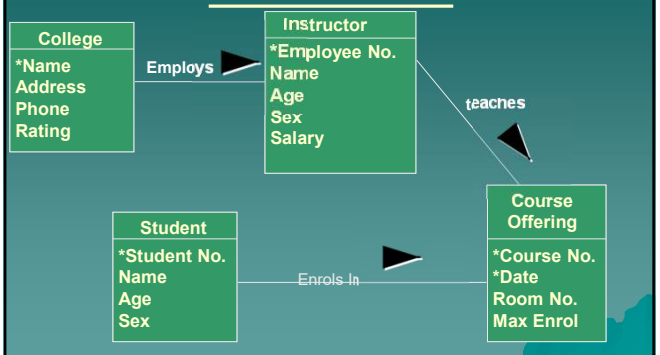


43

43

6 The Entity-Relationship Notation.

Associations

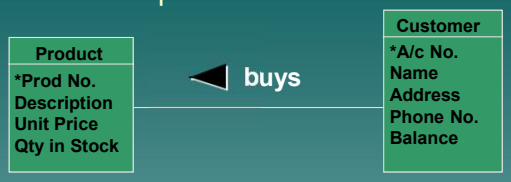


44

44

6 The Entity-Relationship Notation.

More Examples of Associations



Read the sentence in the direction of the arrow:

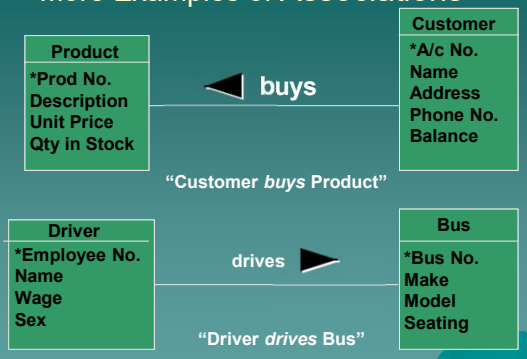
"Customer buys Product."

45

45

6 The Entity-Relationship Notation.

More Examples of Associations



Read the sentence in the direction of the arrow.

46

46

6 The Entity-Relationship Notation.

Multiplicity

Q. When a driver *drives* a bus, how many does she drive?

A. One, usually. But over time?

Why, a whole bunch!

Q. When a Customer *buys* a Product, how many does he buy?

A. One per transaction, perhaps, but over time we hope he'll buy a great many.

47

47

6 The Entity-Relationship Notation.

Multiplicity

- The critical thing we need to know is

Is it **One**, or

Is it **Many**?

- This is the ***Multiplicity*** of the relationship (also called *cardinality*).

48

48

6 The Entity-Relationship Notation.
Multiplicity

- Webster defines Cardinality as "The number of members in a set."
- For us, Multiplicity is the *number of occurrences* of each entity type that can be involved in an association.

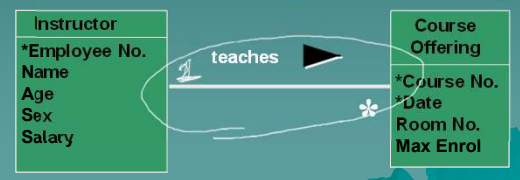
49

49

6 The Entity-Relationship Notation.
Multiplicity

- We diagram this by adding a star (asterisk) below the relationship line whenever it should show "many."
- Read this one as

"Instructor *teaches* many course offerings"



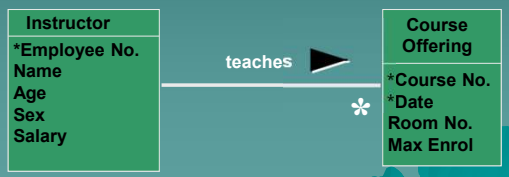
50

50

6 The Entity-Relationship Notation.
Multiplicity

- We refer to this as a

"One-to-Many" Association, or 1:M

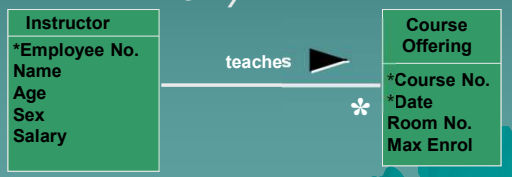


51

51

6 The Entity-Relationship Notation.
Multiplicity

- Read the sentence in the direction of the arrow
- Read the star (asterisk) as the word "*many*."



52

52

6 The Entity-Relationship Notation.
Multiplicity

- And, since we wish many Customers to buy many Products,
- This one is a Many-to-Many association, or M:M

53

53

6 The Entity-Relationship Notation.
Multiplicity

- There are numerous other symbols also used for the "many" end of a relationship,
- including single and double arrows, etc.
- The star (asterisk) is the UML standard.
- Other ERD notations have their own ways to show "many" and what direction to read the sentence.

54

54

⑥ The Entity-Relationship Notation. Summary

- Multiplicity is *one* (straight line) or *many* (asterisk) below each end of the line.

1:M

—————

*

M:M

—————

* *

55

55

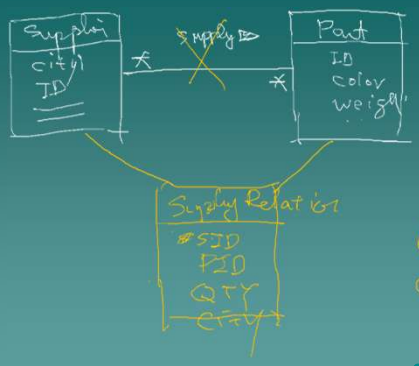
Let's try to draw the problem of supply part again

S#: 零件供應商的編號
SNAME: 零件供應商的姓名
CITY1 零件供應商的城市
P# 零件編號
PNAME 零件名稱
COLOR 零件色彩
WEIGHT 零件重量
CITY2 零件所儲存的城市
QTY 零件的存量

一個零件可能有多個供應商可以供應, 例如正新輪胎與南港輪胎都可以提供米其林10039號輪胎,而正新輪胎可以存放在板橋,但是南港輪胎則庫存在內湖

56

56



57

57

你現在是一個系統分析師, 你的任務是分析一個校務行政系統. 請畫出此系統的UML 圖,
以下是概略的需求分析

- 一個老師可以教許多門課,
- 每一個學生可以自行選修課來修,
- 目前的開授的科目有英文, 數學, 物理, 化學, 國文.
- 每一個開課的科目包含授課時數, 授課教室, 授課時段, 授課年級與班級以及授課老師等資訊.
- 學生的學籍包含身分證號碼, 學號, 年級, 姓名, 住址
- 每個學生都會被編到一個班級上課, 由一位老師擔任導師

58

58

You are a system analyst and you are responsible for analyze an e-school system
Please draw UML diagram for the following requirement

- A teacher can teach many courses
- A student can enroll in several courses
- The course subject include Chinese, English, Math, Chemistry
- A course has hours, classroom, time slots, grades, class, teacher
- A student has ID, name, grade, address
- Each student will be assigned to a class. There is one teacher will be assigned to the class as an advisor.

59

59

Discussion Comment

- Do you need to make subject into a table?
  - If for a hundred year, your subject name change little, a table can be skipped. A string field can be used in the course table. But somehow your program need to maintain a list of these subjects (strings) as well.
  - However, if you have additional attributes needed to be added to a subject. Such as course description, course textbook, etc. Making it into a table is a better solution.

60

60

What if you don't normalize?

- You may write

```
Class student {
   int sid ;
   vector<course*> my_courses;
}

Class  course {
   int cid ;
   vector<students*> students ;
}
```

- What's wrong?
  - 如果上級長官有要求，要求你紀錄每個學生修課時的到課時數 (註: 任何新的屬性與兩者都相關)

61